```
### Cell 1: Imports
begin
    using Pkg
    Pkg.activate(".")
    Pkg.instantiate()
    using InfiniteOpt, Ipopt, Plots, NLsolve, LinearAlgebra
end
```

```
  Activating project at `~/.julia/pluto_notebooks`
```

```
### Cell 2: Parameters (Sun-Earth 2D)
begin
    μ = 3.00348961491547e-6          # Sun = primary, Earth+Moon = secondary
    const u_max = 0.1
    const N_supports = 501
    tf_lower = 50.0
    tf_upper = 400.0
    tf_start = 180.0

    println("μ = ", μ, ", u_max = ", u_max, ", supports = ", N_supports)
end
```

```
μ = 3.00348961491547e-6, u_max = 0.1, supports = 501
```

```
0.9900265839144706
```

```
### Cell 3: Compute L1 (2D is identical)
begin
    function dOmega_dx(x, μ)
        r1 = abs(x + μ)
        r2 = abs(x - (1-μ))
        return x - (1-μ)*(x + μ)/r1^3 - μ*(x - (1-μ))/r2^3
    end
    res = nlsolve(x -> dOmega_dx(x[1], μ), [0.99])
    if !NLsolve.converged(res)
        @warn "L1 solver did not converge"
    end
    x_L1 = res.zero[1]
    println("Sun-Earth L1 at x = ", x_L1)
    x_L1
end
```

```
Sun-Earth L1 at x = 0.9900265839144706
```

```
An InfiniteOpt Model
Feasibility problem with:
  Finite parameters: 0
  Infinite parameter: 1
  Variables: 7
  Derivatives: 0
  Measures: 0
  `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.LessThan{Float64}`: 3 constraints
  `InfiniteOpt.GeneralVariableRef`-in-`MathOptInterface.GreaterThan{Float64}`: 3 constraints
Names registered in the model: tf, ux, uy, vx, vy, x, y, τ, ∂Ωx, ∂Ωy
Transformation backend information:
  Backend type: TranscriptionBackend
  Solver: Ipopt
  Transformation built and up-to-date: false
```

```julia
### Cell 4: Model + variables (2D only)
begin
    model = InfiniteModel(Ipopt.Optimizer)
    @infinite_parameter(model, τ ∈ [0, 1], num_supports = N_supports)
    @variable(model, tf_lower <= tf <= tf_upper, start = tf_start)

    # States (only x,y,vx,vy — z forced to zero)
    @variable(model, x, Infinite(τ), start = τ -> (1-μ + 0.001)*(1-τ) + x_L1*τ)
    @variable(model, y, Infinite(τ), start = τ -> 0.0)
    @variable(model, vx, Infinite(τ), start = τ -> 0.0)
    @variable(model, vy, Infinite(τ), start = τ -> 0.0)

    # Controls (only ux, uy — uz not needed)
    @variable(model, -u_max <= ux <= u_max, Infinite(τ))
    @variable(model, -u_max <= uy <= u_max, Infinite(τ))

    # Potential gradients (2D versions)
    r1(x_,y_) = sqrt((x_ + μ)^2 + y_^2)
    r2(x_,y_) = sqrt((x_ - (1-μ))^2 + y_^2)
    ∂Ωx(x_,y_) = x_ - (1-μ)*(x_ + μ)/r1(x_,y_)^3 - μ*(x_ - (1-μ))/r2(x_,y_)^3
    ∂Ωy(x_,y_) = y_ - (1-μ)*y_/r1(x_,y_)^3 - μ*y_/r2(x_,y_)^3

    model[:∂Ωx] = ∂Ωx
    model[:∂Ωy] = ∂Ωy

    model
end
```

$$\int_{\tau\in[0,1]} \mathbf{abs}\left(ux(\tau)\right) + \mathbf{abs}\left(uy(\tau)\right)d\tau$$

```
### Cell 5: Dynamics + BCs + Objective (2D)
begin
    # Equations of motion
    @constraint(model, ∂(x, τ) == tf * vx)
    @constraint(model, ∂(y, τ) == tf * vy)
    @constraint(model, ∂(vx, τ) == tf * (2*vy + model[:∂Ωx](x,y) + ux))
    @constraint(model, ∂(vy, τ) == tf * (-2*vx + model[:∂Ωy](x,y) + uy))

    # Initial: just outside Earth, zero velocity, in the x-axis
    @constraint(model, x(0) == 1 - μ + 0.001)
    @constraint(model, y(0) == 0.0)
    @constraint(model, vx(0) == 0.0)
    @constraint(model, vy(0) == 0.0)

    # Final: exactly at L1, zero velocity
    @constraint(model, x(1) == x_L1)
    @constraint(model, y(1) == 0.0)
    @constraint(model, vx(1) == 0.0)
    @constraint(model, vy(1) == 0.0)

    # Minimum L1-fuel (bang-coast-bang style)
    @objective(model, Min, ∫(abs(ux) + abs(uy), τ))
    # or use quadratic: @objective(model, Min, ∫(ux^2 + uy^2, τ))
end
```

```
### Cell 6: Solve
begin
    set_silent(model)
    @time optimize!(model)
    println("Optimization done!")
end
```

```
 24.477409 seconds (2.63 M allocations: 132.930 MiB, 0.07% gc time, 0.90% compi
lation time)
Optimization done!
```

```
([1.00099, 1.00774, 1.0342, 1.06495, 1.08247, 1.08552, 1.07794, 1.05034, 0.991944,    more ,0.
```

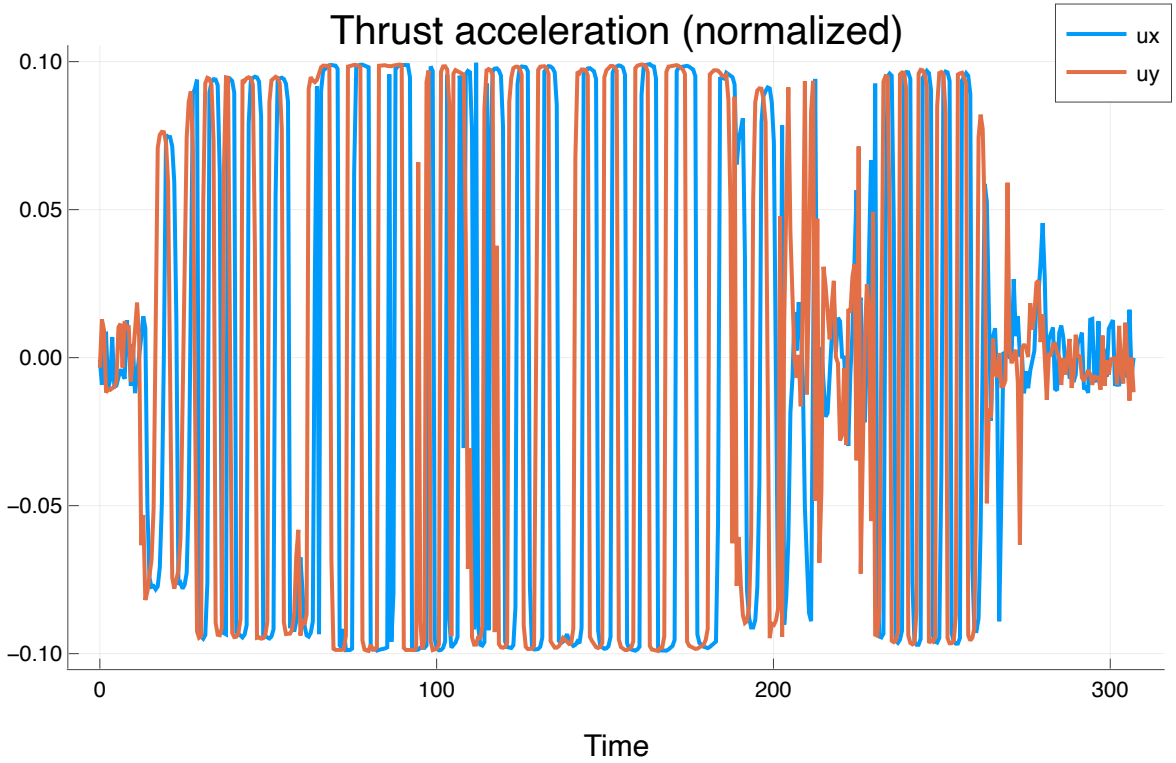
```
### Cell 7: Extract solution
begin
    opt_x  = value(x)
    opt_y  = value(y)
    opt_vx = value(vx)
    opt_vy = value(vy)
    opt_ux = value(ux)
    opt_uy = value(uy)
    opt_tf = value(tf)
    τ_grid = value.(supports(τ))
    t_grid = τ_grid .* opt_tf

    println("Transfer time = ", round(opt_tf, digits=2), " normalized units")
    println("              ≈ ", round(opt_tf * 58.1328, digits=1), " days")
    println("              ≈ ", round(opt_tf * 58.1328 / 365.25, digits=2), " years")

    solution = (opt_x, opt_y, opt_vx, opt_vy, opt_ux, opt_uy, t_grid, opt_tf)
end
```

```
Transfer time = 306.94 normalized units
              ≈ 17843.1 days
              ≈ 48.85 years
```

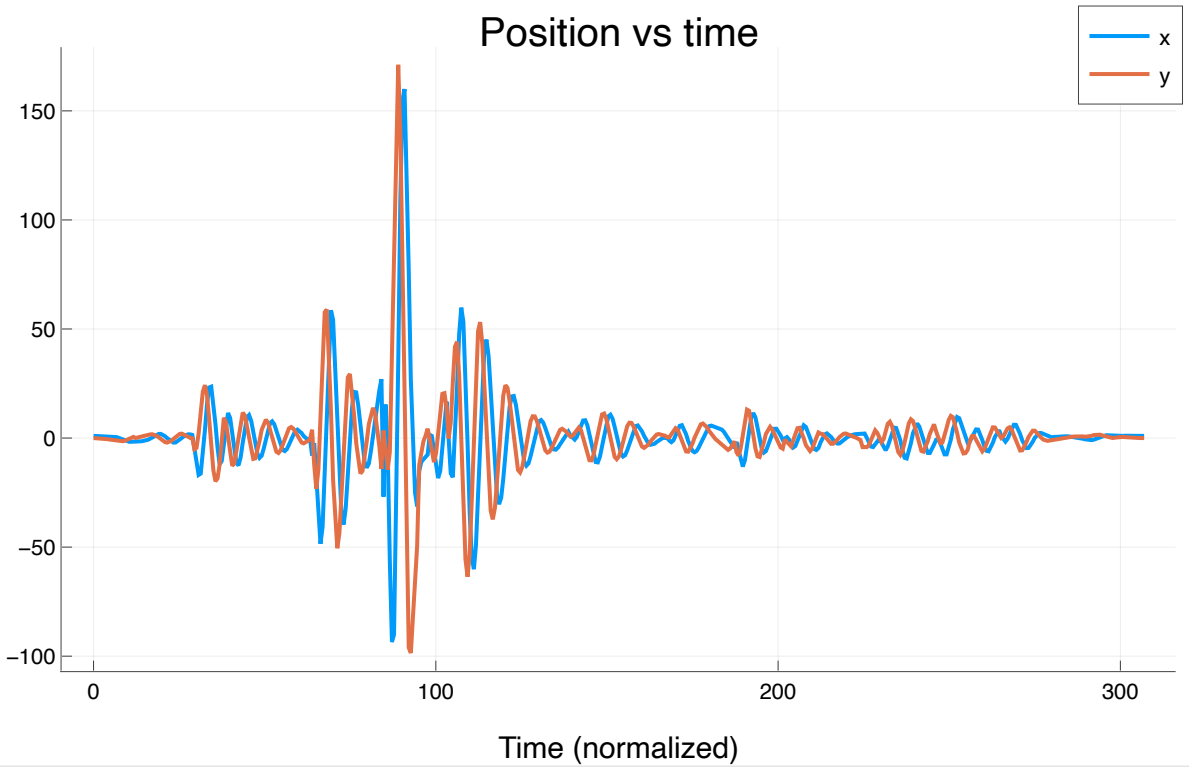
```julia
### Cell 8: plots
begin

    # Main trajectory
    p_traj = plot(opt_x, opt_y, lw=2.5, label="Optimal low-thrust trajectory",
                  title="Earth → Sun-Earth L1 (planar CR3BP)", aspect_ratio=:equal,
                  xlabel="x", ylabel="y", color=:purple)
    scatter!(p_traj, [0.0],   [0], marker=:circle,  markersize=10, label="Sun")
    scatter!(p_traj, [1-μ],   [0], marker=:star5,   markersize=8,  label="Earth")
    scatter!(p_traj, [x_L1],  [0], marker=:diamond, markersize=9,  label="L1")

    # Control history
    p_ctrl = plot(t_grid, [opt_ux opt_uy], label=["ux" "uy"], lw=2,
                  title="Thrust acceleration (normalized)", xlabel="Time")

    # State history
    p_states = plot(t_grid, [opt_x opt_y], label=["x" "y"], lw=2,
                    title="Position vs time", xlabel="Time (normalized)")

    display(p_traj)
    display(p_ctrl)
    display(p_states)
end
```

```
markershape star5 is unsupported with Plots.PlotlyBackend(). Choose from: [:auto, :
circle, :cross, :diamond, :dtriangle, :hexagon, :hline, :none, :octagon, :pentagon,
:rect, :utriangle, :vline, :x, :xcross]
```

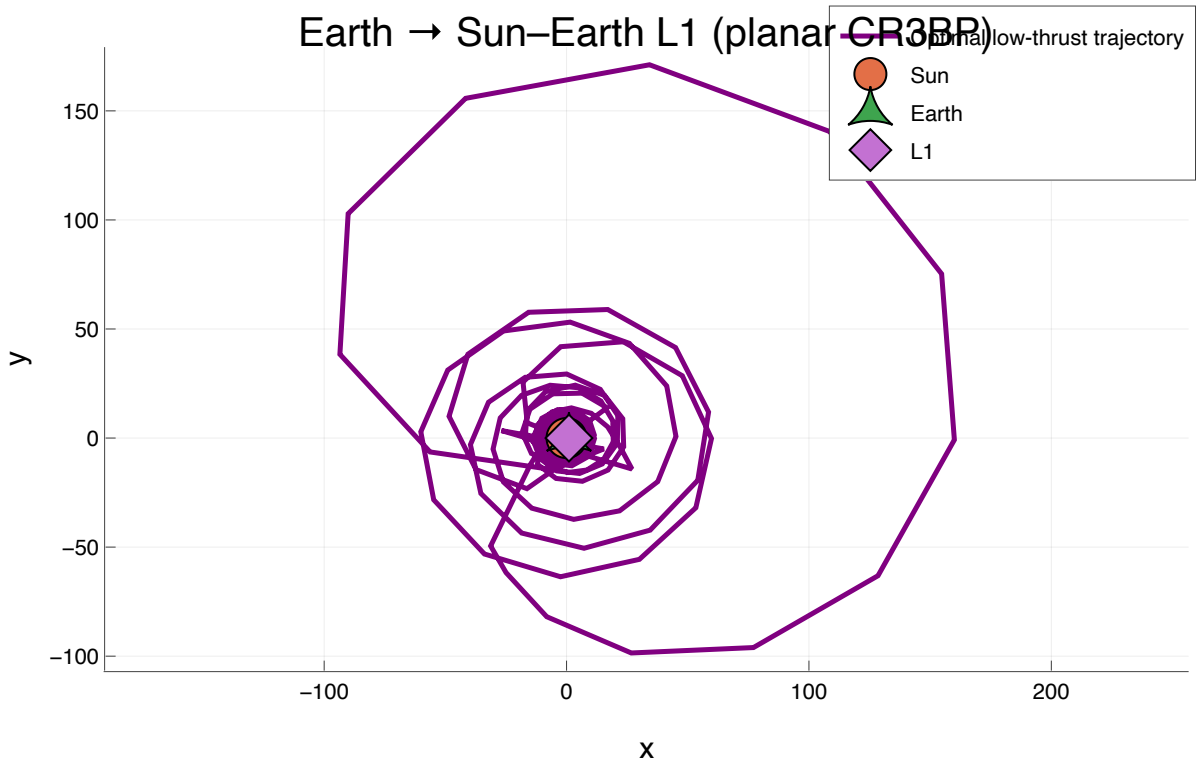
```
Plot{Plots.PlotlyBackend() n=4}
Plot{Plots.PlotlyBackend() n=2}
Plot{Plots.PlotlyBackend() n=2}
```


Thrust acceleration (normalized)
ux
uy
0.10
0.05
0.00
−0.05
−0.10
0
100
200
300
Time

```
1 p_ctrl
```


Position vs time
x
y
150
100
50
0
−50
−100
0
100
200
300
Time (normalized)

```
1 p_states
```


Earth → Sun–Earth L1 (planar CR3BP)
Optimal low-thrust trajectory
Sun
Earth
L1
150
100
50
0
−50
−100
−100
0
100
200
y
x

```
1 p_traj
```